

```
# --- LIENZO 4: LISTADO DE DATOS MENSUALES ---
```

```
with st.expander("📁 **LISTADO DE DATOS MENSUALES**", expanded=True):  
    st.subheader("📋 Listado Completo de Datos Mensuales")
```

```
# Verificar si hay datos en el dataset anual
```

```
if df_mensual.empty or len(df_mensual) == 0:  
    st.warning("No hay datos disponibles en el dataset mensual.")
```

```
else:
```

```
    # Filtros por mes y Año
```

```
    col1, col2 = st.columns(2)
```

```
with col1:
```

```
    # Obtener meses disponibles ordenados
```

```
    orden_meses = [  
        "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",  
        "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"
```

```
    ]
```

```
# Filtrar solo los meses que existen en el DataFrame
```

```
meses_disponibles = df_mensual["Mes"].dropna().unique()
```

```
# Ordenar los meses según el orden natural
```

```
meses_disponibles = sorted(  
    [m for m in meses_disponibles if m in orden_meses],  
    key=lambda x: orden_meses.index(x)  
)
```

```
mes_seleccionado = st.selectbox(  
    "📅 Seleccione el mes:",  
    options=meses_disponibles,  
    key="mes_mensual",  
    index=meses_disponibles.index("Enero") if "Enero" in meses_disponibles else 0  
)
```

```
with col2:
```

```
    # Obtener Años disponibles ordenados
```

```
    if 'Año' in df_mensual.columns:
```

```
        Años_disponibles = sorted(df_mensual['Año'].unique())
```

```
        Año_seleccionado = st.selectbox(  
            "Selecciona el Año",  
            options=Años_disponibles,  
            key="anio_mensual",  
            index=len(Años_disponibles)-1 if len(Años_disponibles) > 0 else 0  
)
```

```
    else:
```

```
        st.error("No se encontró la columna 'Año' en los datos")
```

```
Año_seleccionado = None
```

```
# Aplicar filtros
```

```
if mes_seleccionado and Año_seleccionado and 'Mes' in df_mensual.columns and 'Año' in df_mensual.columns:
```

```
    df_filtradomensual = df_mensual[
        (df_mensual['Mes'] == mes_seleccionado) &
        (df_mensual['Año'] == Año_seleccionado)
    ].copy()
```

```
else:
```

```
    df_filtradomensual = df_mensual.copy()
```

```
# Ordenar por Indicador de mayor a menor si la columna existe
```

```
if 'Indicador' in df_filtradomensual.columns:
```

```
    df_filtradomensual = df_filtradomensual.sort_values('Indicador', ascending=False)
```

```
else:
```

```
    st.warning("No se encontró la columna 'Indicador' para ordenar")
```

```
# Mostrar el listado de datos
```

```
st.subheader("📊 Listado Detallado (Ordenado por Indicador - Mayor a Menor)")
```

```
# Columnas a mostrar
```

```
columnas_a_mostrar = ['CodPpdd', 'Ppdd', 'Oportunos', 'Total_Fuas', 'Indicador']
```

```
# Verificar que las columnas existan en el DataFrame
```

```
columnas_existentes = [col for col in columnas_a_mostrar if col in df_filtradomensual.columns]
```

```
if columnas_existentes:
```

```
    # Preparar DataFrame para mostrar
```

```
    df_mostrar_mensual = df_filtradomensual[columnas_existentes].copy().head(23)
```

```
    df_mostrar_mensual = df_mostrar_mensual.reset_index(drop=True)
```

```
    df_mostrar_mensual.insert(0, "Nº", range(1, len(df_mostrar_mensual) + 1))
```

```
# LIMPIAR DATOS NUMÉRICOS
```

```
columnas_numericas = ["Nº", "CodPpdd", "Oportunos", "Total_Fuas", "Indicador"]
```

```
df_mostrar_mensual = limpiar_y_preparar_dataframe(df_mostrar_mensual, columnas_numericas)
```

```
# Crear una versión para mostrar con el indicador formateado como porcentaje
```

```
df_mostrar_formateado = df_mostrar_mensual.copy()
```

```
if 'Indicador' in df_mostrar_formateado.columns:
```

```
    df_mostrar_formateado['Indicador'] = df_mostrar_formateado['Indicador'].apply(
        lambda x: f"{x:.1%}" if isinstance(x, (int, float)) and not pd.isna(x) else "N/A"
    )
```

```
# APLICAR ESTILOS CON COLORES AL INDICADOR
```

```
cell_style_indicador = JsCode("""
```

```
function(params) {
```

```

if (params.value === 'N/A' || params.value === null || params.value === undefined) {
    return {backgroundColor: 'lightgray', color: 'black', fontWeight: 'bold', textAlign: 'center'};
}

var val = parseFloat(params.value.toString().replace('%', '')) / 100;

if (val >= 0.75) {
    return {backgroundColor: '#00b050', color: 'white', fontWeight: 'bold', textAlign: 'center',fontSize: '19px'};
} else if (val >= 0.60) {
    return {backgroundColor: '#ffcc66', color: 'black', fontWeight: 'bold', textAlign: 'center',fontSize: '19px'};
} else if (val >= 0.25) {
    return {backgroundColor: '#ff7c80', color: 'black', fontWeight: 'bold', textAlign: 'center',fontSize: '19px'};
} else {
    return {backgroundColor: '#ff0000', color: 'white', fontWeight: 'bold', textAlign: 'center',fontSize: '19px'};
}
}
""")

```

MOSTRAR SOLO LA TABLA CON COLORES

```

from st_aggrid import AgGrid, GridOptionsBuilder, GridUpdateMode

```

```

# --- build grid options ---

```

```

gb = GridOptionsBuilder.from_dataframe(df_mostrar_formateado)

```

```

gb.configure_default_column(
    resizable=True,
    sortable=True,
    filter=True,
    wrapText=True,
    autoHeight=True,
    cellStyle={"fontSize": "18px", "fontFamily": "Segoe UI"},
    headerClass="custom-header" # clase que aplicaremos al header
)

```

```

# ajustar alturase

```

```

gb.configure_grid_options(rowHeight=32, headerHeight=44, domLayout='normal')

```

```

#Color Columna Indicador

```

```

gb.configure_column(
    "Indicador",
    cellStyle=cell_style_indicador,
    valueFormatter=JsCode("(function(params) { if (!params.value) return ''; return params.value; })")
)

```

#Color Columna Indicador

```
grid_options = gb.build()
```

--- CSS que se inyecta DENTRO del componente (custom_css) ---

```
custom_css = {  
    # variables del tema (afectan todo el grid, buen fallback)  
    ".ag-theme-streamlit": {  
        "--ag-font-size": "20px",  
        "--ag-header-height": "44px"  
    },  
    # selectores directamente sobre header (por compatibilidad)  
    ".ag-header-cell-label": {  
        "font-size": "20px",  
        "font-weight": "600",  
        "font-family": "Segoe UI"  
    },  
    ".ag-header-cell-text": {  
        "font-size": "20px",  
        "font-weight": "600"  
    },  
    # clase que asignamos con headerClass  
    ".custom-header": {  
        "font-size": "20px",  
        "font-weight": "600"  
    }  
}
```

--- render AgGrid y pasar custom_css ---

```
AgGrid(  
    df_mostrar_formateado,  
    gridOptions=grid_options,  
    height=600,  
    auto_size_columns=True,      # 🔄 SOLO ESTA  
    update_mode=GridUpdateMode.NO_UPDATE,  
    enable_enterprise_modules=False,  
    theme='streamlit',  
    custom_css=custom_css,  
    allow_unsafe_jscode=True  
)
```

```
st.markdown("""  
    <style>  
    .custom-header {  
        font-size: 20px !important;  
        font-weight: 600 !important;
```

```
font-family: 'Segoe UI', sans-serif !important;
}
</style>
"""', unsafe_allow_html=True)
```

Fin Tabla con Colores

Leyenda de colores

```
st.markdown("""
** 📊 Leyenda de Indicadores:**

- <span style='color: white; background-color: #00b050; padding: 2px 6px; border-radius: 3px; font-weight:
bold;'>Bueno (75-100%)</span>

- <span style='color: black; background-color: #ffcc66; padding: 2px 6px; border-radius: 3px; font-weight:
bold;'>Regular (60-74%)</span>

- <span style='color: black; background-color: #ff7c80; padding: 2px 6px; border-radius: 3px; font-weight: bold;'>En
Proceso (25-59%)</span>

- <span style='color: white; background-color: #ff0000; padding: 2px 6px; border-radius: 3px; font-weight:
bold;'>Malo (0-24%)</span>

- <span style='color: black; background-color: lightgray; padding: 2px 6px; border-radius: 3px; font-weight:
bold;'>Datos Inválidos</span>

""", unsafe_allow_html=True)
```

Mostrar información adicional

```
st.info(f"📄 Mostrando {len(df_mostrar_mensual)} registros del mes {mes_seleccionado} del Año
{Año_seleccionado}, ordenados por Indicador (mayor a menor).")
```

else:

```
st.error("No se encontraron las columnas especificadas en el dataset.")
st.write("Columnas disponibles en el dataset:", list(df_filtradomensual.columns))
```